

A continuación tienes una guía **paso a paso, clara y completa**, para **desplegar un proyecto Django en producción en una red local**, utilizando una computadora con **Windows como servidor** con la IP **192.162.1.193**.



Desplegar Django en Producción (Windows – Red Local)



Requisitos Previos

- Proyecto Django funcional
 - Python instalado (recomendado 3.10+)
 - Pip instalado
 - Una PC que será el servidor con IP **192.162.1.193**
 - Todos los PC clientes en la misma red
-



1 Configurar el Proyecto Django



✓ Editar settings.py

Busca la línea:

```
ALLOWED_HOSTS = []
```

Cámbiala por:

```
ALLOWED_HOSTS = ['192.162.1.193', 'localhost', '127.0.0.1']
```



Si luego accedes desde otro PC, asegúrate de que su IP esté permitida o usa `'*'` para toda la red:

```
ALLOWED_HOSTS = ['*']
```



✓ Activar archivos estáticos

En `settings.py`:

```
STATIC_URL = '/static/' STATIC_ROOT = BASE_DIR / 'staticfiles'
```

Luego ejecuta:

```
python manage.py collectstatic
```

2 Crear entorno virtual (recomendado)

En la carpeta del proyecto:

```
python -m venv venv
```

Activar:

```
venv\Scripts\activate
```

Instalar dependencias:

```
pip install -r requirements.txt
```

3 Instalar un servidor de producción: Waitress

Waitress funciona perfecto en Windows.

```
pip install waitress
```

4 Crear archivo para ejecutar con Waitress

Crea un archivo llamado `server.py` afuera o dentro del proyecto:

```
from waitress import serve from your_project_name.wsgi import application #  
Cambia 'your_project_name' serve(application, host='192.162.1.193', port=8000)
```

👉 Sustituye `your_project_name` por el nombre de tu carpeta principal del proyecto.

Ejemplo:

```
from waitress import serve from Boutique.wsgi import application  
serve(application, host='192.162.1.193', port=8000)
```

Ejecuta:

```
python server.py
```

5 Abrir el puerto en Windows Firewall

 Panel de control → Firewall → Configuración avanzada → Reglas de entrada

- Nueva regla
- Tipo: **Puerto**
- TCP → Puerto 8000
- Permitir conexión
- Aplicar a todos los perfiles
- Nombre: **Django 8000**

6 Probar en la red

Desde otra computadora en la misma red abre:

```
http://192.162.1.193:8000
```

 Si aparece tu app, ¡ya estás en producción local!

7 Opcional: Ejecutar como Servicio (para que no se cierre)

Puedes usar herramientas como:

- **NSSM (Non-Sucking Service Manager)** para ejecutar Python como servicio

 <https://nssm.cc/> ➞

O crear un .bat:

```
cd C:\ruta\de\tu\proyecto venv\Scripts\activate python server.py
```

Colocarlo en Inicio de Windows.



Recomendaciones Finales

Tema	Recomendación
Base de datos	Usar PostgreSQL si hay muchos usuarios
Seguridad	Cambia SECRET_KEY en producción
Debug	En settings.py : DEBUG = False
Servidor web	Puedes usar Nginx/Apache como proxy si crece el sistema



¿Quieres que te prepare un script automatizado para el despliegue (todo listo con un clic)?

- Sí
- No
- Sí, pero con PostgreSQL



Responde con tu opción.

Si

¡Perfecto! Te voy a crear un **script automatizado** para que puedas ejecutar tu proyecto Django en **producción local con un solo clic en Windows**, usando **Waitress** como servidor.



SCRIPT AUTOMÁTICO PARA DESPLEGAR DJANGO EN WINDOWS (PRODUCCIÓN LOCAL)

📌 Este script hará:

- ✓ Activar el entorno virtual
 - ✓ Ejecutar el servidor Waitress con tu IP
 - ✓ Mantener la ventana activa para ver errores
 - ✓ Preparado para ejecutar con doble clic
-

📌 PASO 1: Crear archivo `server.py`

📌 En la carpeta del proyecto, crea el archivo **server.py** con el siguiente contenido

👉 No olvides cambiar `NOMBRE_PROYECTO` por el nombre real de tu proyecto Django.

```
from waitress import serve from NOMBRE_PROYECTO.wsgi import application # <--
Cambiar serve( application, host='192.162.1.193', # IP del servidor port=8000
)
```

📌 Ejemplo si tu proyecto se llama *BoutiqueApp*:

```
from waitress import serve from BoutiqueApp.wsgi import application
serve(application, host='192.162.1.193', port=8000)
```

📌 PASO 2: Crear archivo ejecutable `iniciar_server.bat`

📌 Crea un archivo llamado **iniciar_server.bat** en la misma carpeta del proyecto:

```
@echo off title Servidor Django en Producción - Waitress echo
===== echo INICIANDO SERVIDOR DJANGO EN
LOCAL echo ===== echo. REM Activar
entorno virtual call venv\Scripts\activate REM Ejecutar servidor con Waitress
python server.py echo. echo ----- echo
PRESIONA UNA TECLA PARA CERRAR EL SERVIDOR echo -----
----- pause >nul
```

📌 PASO 3: Ejecutar con doble clic

- Solo haz doble clic en **iniciar_server.bat**
- Accede desde cualquier PC de tu red:

`http://192.162.1.193:8000`



RECORDATORIO IMPORTANTE DE SEGURIDAD

En tu `settings.py` :

```
DEBUG = False ALLOWED_HOSTS = ['192.162.1.193', 'localhost', '127.0.0.1']
```



OPCIONAL (RECOMENDADO): Colectar archivos estáticos

Si aún no lo hiciste:

```
python manage.py collectstatic
```



¡Listo! Ya tienes un servidor Django en producción con un clic.



¿Deseas ahora agregar uno de estos extras?



Elige uno y te lo configuro:

- 1 Ejecutar como **Servicio de Windows** (se enciende solo al prender la PC)
- 2 Instalación automática de **PostgreSQL** y configuración de Django
- 3 Configurar **HTTPS con certificado SSL** (solo en la red local)
- 4 Crear interfaz web para gestionar backups de la BD



Escribe el número de la opción que deseas.

¡Excelente! Vamos a configurar tu servidor Django para que se **inicie automáticamente como Servicio de Windows**, usando una herramienta profesional llamada **NSSM** (Non-Sucking Service Manager).

Esto permitirá que tu app Django:

- ✓ Se inicie sola al encender la computadora
- ✓ No necesite abrir ventanas ni .bat
- ✓ Funcione en segundo plano como un servidor real

1) Instalar NSSM en Windows

 Descarga NSSM (versión estable):

 <https://nssm.cc/download> ↗

 Luego:

1. Extrae el ZIP
2. Copia la carpeta a `C:\nssm\` (crea la carpeta si no existe)

 Ejemplo final:

`C:\nssm\nssm.exe`

2) Crear el servicio para el Proyecto Django

 Abre Símbolo del sistema (CMD) como Administrador y ejecuta:

```
C:\nssm\nssm.exe install DjangoServidor
```

Se abrirá una ventana de configuración del servicio.

3) Configurar NSSM

Completar los campos así:

Application Path

👉 Selecciona `python.exe` de tu entorno virtual:

`C:\ruta\del\proyecto\venv\Scripts\python.exe`

Startup directory

Ruta de tu proyecto Django:

`C:\ruta\del\proyecto\`

Arguments

Introduce:

`server.py`

📌 Esto hará que Windows ejecute tu script con Waitress.



4) Configurar reinicio automático del servicio

Ir a pestaña **Shutdown → Restart**

✓ Marcar "Restart" para evitar caídas por errores.



5) Guardar y arrancar el servicio

📌 En CMD (como Admin):

```
net start DjangoServidor
```



6) Probar desde otra PC

Abre en navegador otra computadora de la red:

`http://192.162.1.193:8000`



Si carga, el servidor está ahora en modo producción y automático.



7) Manejo del servicio

Acción	Comando
Detener servidor	<code>net stop DjangoServidor</code>
Iniciar servidor	<code>net start DjangoServidor</code>
Quitar servicio	<code>C:\nssm\nssm.exe remove DjangoServidor confirm</code>



Recomendación Extra (Opcional)

💡 Si tu servidor será usado por varios usuarios concurrentes, te recomiendo usar **PostgreSQL** en lugar de SQLite.

✚ Si deseas, puedo:

☒ **Instalar PostgreSQL + Configurar Django + Crear usuario seguro + Backup automático**

👉 ¿Deseas que lo configure?

Sí / No / Más adelante

Perfecto 😊. Cuando estés listo solo me dices **Sí**, **No**, o **Más adelante**, y te lo configuro.

Mientras tanto, ya puedes usar tu servicio Django en producción local sin abrir nada manualmente. Si necesitas:

- Monitoreo del servicio
- Logs automáticos
- Actualizaciones sin detener usuarios
- Backup automático
- HTTPS en red local

Solo pídelo y te guío paso a paso. 💪 💻

¿Deseas agregar logs del servidor para ver errores y actividad?



Recomendado en producción

👉 Sí / No / Después

lo probe en un equipo movil y la pagina perdio toda la configuracion

Cuando la página se ve sin estilos en otro dispositivo (móvil, otra PC, tablet), significa que los archivos estáticos (CSS, JS, imágenes) no están siendo servidos correctamente en producción.

Esto es normal con Django en producción: Django NO sirve archivos estáticos automáticamente.

✅ SOLUCIÓN: Servir archivos estáticos correctamente

1 Revisa tu `settings.py`

Asegúrate que tengas así:

```
DEBUG = False STATIC_URL = '/static/' STATIC_ROOT = BASE_DIR / 'staticfiles'
```

👉 Si ya existe `STATIC_ROOT`, no lo dupliques.

2 Ejecuta la recolección de archivos

Abre CMD en la carpeta del proyecto (activar venv si es necesario):

```
python manage.py collectstatic
```

Esto creará una carpeta:

```
/staticfiles/
```

Aquí irán todos los CSS, JS, imágenes de tus apps y librerías como Bootstrap.

3 Servir estáticos con Waitress (solo temporal)

⚠️ Waitress NO sirve estáticos en producción, entonces debes agregar una solución temporal con `whitenoise`.

✂️ Instalar whitenoise:

```
pip install whitenoise
```

4 Configurar Whitenoise en `settings.py`

Busca:

```
MIDDLEWARE = [
```

Agrega después de `SecurityMiddleware`:

```
'middleware.security.SecurityMiddleware',  
'whitenoise.middleware.WhiteNoiseMiddleware',
```

Ejemplo completo:

```
MIDDLEWARE = [ 'django.middleware.security.SecurityMiddleware',  
'whitenoise.middleware.WhiteNoiseMiddleware',  
'django.contrib.sessions.middleware.SessionMiddleware',  
'django.middleware.common.CommonMiddleware',  
'django.middleware.csrf.CsrfViewMiddleware',  
'django.contrib.auth.middleware.AuthenticationMiddleware',  
'django.contrib.messages.middleware.MessageMiddleware',  
'django.middleware.clickjacking.XFrameOptionsMiddleware', ]
```

5 Habilitar compresión para estáticos

En el mismo `settings.py`, agrega:

```
STATICFILES_STORAGE =  
"whitenoise.storage.CompressedManifestStaticFilesStorage"
```

Recolectar otra vez

```
python manage.py collectstatic
```

Reiniciar el servicio en Windows

```
net stop DjangoServidor net start DjangoServidor
```

Resultado esperado

Ahora en cualquier dispositivo en la red (móvil o PC) tendrás tu app con estilos completos:

- ✓ Bootstrap
 - ✓ CSS propio
 - ✓ Imágenes
 - ✓ JS
-

BONUS: ¿Tu sitio usa rutas absolutas de imágenes en CSS?

Si usas recursos con rutas como:

```
background-image: url("/static/img/bg.jpg");
```

Todo funcionará bien.

Pero si usas rutas relativas incorrectas como:

```
background-image: url("img/bg.jpg");
```

 Eso debes corregir.

 ¿Quieres que revise tu estructura estática  para asegurar que todo quede perfecto?

Puedes responder: Sí y me envías una captura de tu proyecto (carpetas), o:

👤 Solo la estructura, por ejemplo:

```
proyecto/  
  app/  
  static/  
  templates/
```

👉 ¿Deseas que te revise la estructura?

Sí / No / Más tarde

ya funciona me puedes crear un readme con todos estos pasos para poner en produccion el proyecto de django, gracias

¡Excelente! Aquí tienes un **README profesional** para incluir en tu proyecto Django con todos los pasos para ponerlo en producción en una **red local con Windows + Waitress + Whitenoise + Servicio automático con NSSM**.

📌 Deploy de Proyecto Django en Producción Local (Windows)

Este documento describe cómo desplegar un sistema Django en **modo producción en una red local**, usando:

- **Windows** como servidor
- **Waitress** como servidor WSGI
- **Whitenoise** para archivos estáticos
- **NSSM** para ejecutar el proyecto como servicio de Windows (autoarranque)
- **Acceso desde PCs y móviles en la red local**

IP utilizada en este ejemplo: 192.162.1.193

Esta IP puede cambiar según tu red. Debes ajustar los valores cuando sea necesario.



1. Configuración de Django



✎ Editar settings.py

```
DEBUG = False ALLOWED_HOSTS = ['192.162.1.193', 'localhost', '127.0.0.1']
STATIC_URL = '/static/' STATIC_ROOT = BASE_DIR / 'staticfiles'
STATICFILES_STORAGE =
"whitenoise.storage.CompressedManifestStaticFilesStorage"
```



✎ Agregar Whitenoise al middleware

```
MIDDLEWARE = [ 'django.middleware.security.SecurityMiddleware',
'whitenoise.middleware.WhiteNoiseMiddleware',
'django.contrib.sessions.middleware.SessionMiddleware',
'django.middleware.common.CommonMiddleware',
'django.middleware.csrf.CsrfViewMiddleware',
'django.contrib.auth.middleware.AuthenticationMiddleware',
'django.contrib.messages.middleware.MessageMiddleware',
'django.middleware.clickjacking.XFrameOptionsMiddleware', ]
```



2. Instalar dependencias

```
pip install waitress whitenoise
```

Si es primera vez configurando el entorno virtual:

```
python -m venv venv venv\Scripts\activate pip install -r requirements.txt
```



3. Recolectar archivos estáticos

```
python manage.py collectstatic
```



4. Crear servidor con Waitress

Crear archivo `server.py` en el proyecto:

```
from waitress import serve from NOMBRE_PROYECTO.wsgi import application #  
Cambiar por tu proyecto serve( application, host='192.162.1.193', port=8000 )
```

✦ Ejemplo si el proyecto se llama BoutiqueApp :

```
from waitress import serve from BoutiqueApp.wsgi import application  
serve(application, host='192.162.1.193', port=8000)
```

5. Script para ejecutar manualmente (opcional)

Crear archivo `iniciar_server.bat` :

```
@echo off title Servidor Django en Producción - Waitress call  
venv\Scripts\activate python server.py pause >nul
```

Puedes ejecutarlo con **dobles clic**.

6. Abrir el puerto en Firewall de Windows

1. Abrir Firewall de Windows
2. Crear Regla de entrada
3. Tipo: **Puerto**
4. Puerto TCP: `8000`
5. Permitir conexión
6. Aplicar a todos los perfiles
7. Nombre recomendado: `Django 8000`

7. Ejecutar como servicio de Windows (Autoarranque)

✦ Instalar NSSM

Descargar desde: <https://nssm.cc/download> ↗

Extraer y colocar en:

C:\nssm\

 Crear servicio

Ejecutar en **CMD como administrador**:

```
C:\nssm\nssm.exe install DjangoServidor
```

Configurar los campos:

Campo	Valor
Application Path	C:\ruta\del\proyecto\venv\Scripts\python.exe
Startup directory	C:\ruta\del\proyecto\
Arguments	server.py

 En pestaña **Shutdown**, activar **Restart**.

 Iniciar servicio

```
net start DjangoServidor
```

 8. Acceso desde la red

En cualquier móvil, laptop o PC en la misma red WiFi/LAN:

```
http://192.162.1.193:8000
```

 9. Comandos de administración del servicio

Acción	Comando
Iniciar	net start DjangoServidor
Detener	net stop DjangoServidor
Remover servicio	C:\nssm\nssm.exe remove DjangoServidor confirm

Notas importantes

- Si la página se ve sin estilos, ejecutar nuevamente:

```
python manage.py collectstatic net stop DjangoServidor net start DjangoServidor
```

- No usar SQLite para entornos con múltiples usuarios.
Recomendado: Migrar a PostgreSQL si el sistema crece.